

SHARING SESSION · POSTGRESQL

# Kenapa Koneksi PostgreSQL Itu Mahal

Dan gimana PgBouncer ngatasinnya pas aplikasi di-scale

— Berangkat dari **satu bug** yang bikin pusing

## GEJALA

# Berawal dari satu bug

Di halaman create:

- `INSERT` sukses
- Data ke-return lengkap sama `id`-nya
- Halaman detail → datanya nggak ada
- Cek ke database → memang nggak ada

// dan yang bikin pusing

**Intermittent.**

**Kadang masuk, kadang nggak.**

Nggak ada error. Nggak ada exception. Kodenya udah bener.

YANG AKAN KITA BAHAS

# Agenda

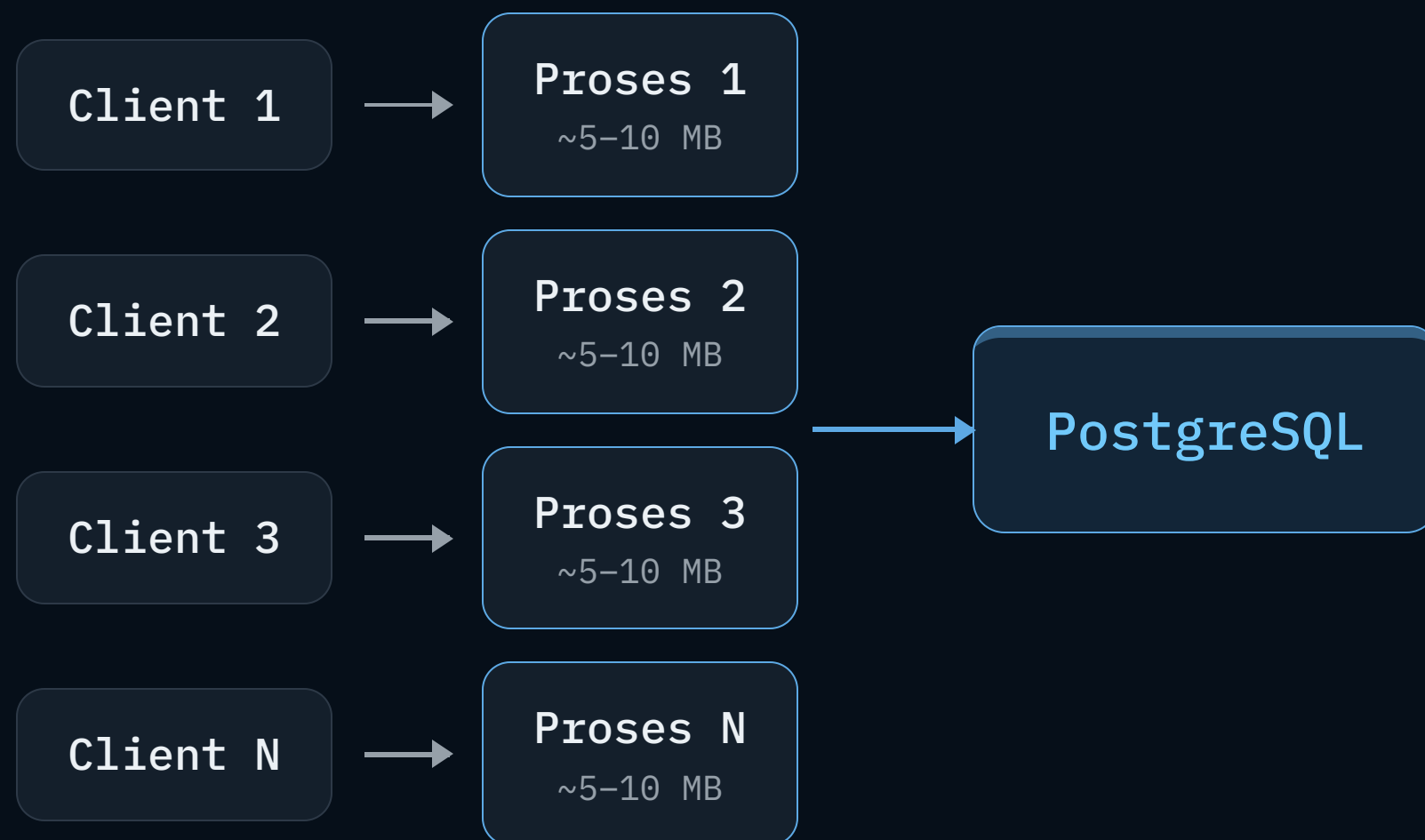
- 1 Kenapa koneksi PostgreSQL mahal
- 2 Ini cuma masalah Postgres atau enggak?
- 3 Jebakannya pas aplikasi scale-out
- 4 Cara kerja PgBouncer
- 5 Konsekuensinya — dan bug di awal terjawab
- 6 Ringkasan praktis

---

BAGIAN 1

# Kenapa satu koneksi PostgreSQL itu ”mahal”?

# Process-per-connection



- Tiap koneksi = **satu proses OS**
- Makan beberapa MB RAM + overhead context-switching
- `max_connections` default cuma **100**



# 2

---

BAGIAN 2

# Ini cuma masalah PostgreSQL?

# Tergantung model koneksinya

| Database             | Model koneksi            | Biaya               | Butuh pooler?        |
|----------------------|--------------------------|---------------------|----------------------|
| PostgreSQL           | Process-per-connection   | Mahal               | Sangat perlu         |
| MySQL                | Thread-per-connection    | Sedang              | Perlu di skala besar |
| MongoDB              | Thread + pool di driver  | Sedang              | Otomatis di driver   |
| Cassandra / Scylla   | Async multiplexed        | Murah               | Enggak               |
| Redis                | Single-thread event loop | Murah               | Opsional             |
| DynamoDB / Firestore | Stateless HTTP           | Nggak ada persisten | Nggak relevan        |

APA YANG MENENTUKAN

# Dua hal yang menentukan

Faktor 1 · Arsitektur engine



makin murah →

Faktor 2 · Pola akses



makin sering buka-tutup →

**Faktor 1** → seberapa mahal satu koneksi

**Faktor 2** → seberapa banyak koneksi dibuka-tutup



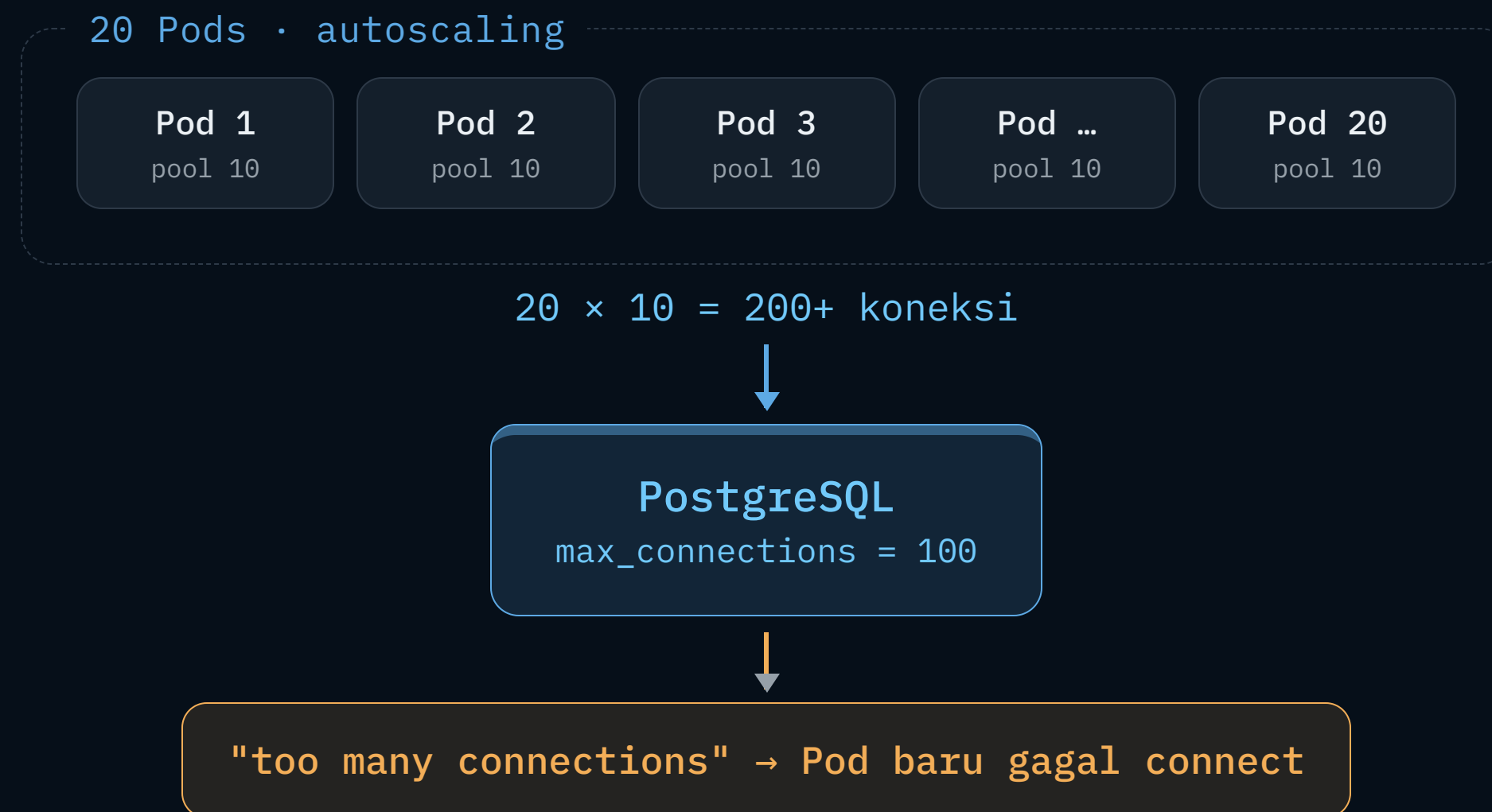
# 3

---

BAGIAN 3

## Jebakannya pas aplikasi **scale-out**

# Hitungan yang menjebak



- 20 pod  $\times$  10 = **200 koneksi** → lewat plafon
- Spike lebih gede → 500 → 1000...
- Ironinya: pas trafik tinggi, autoscaling malah **memperburuk**

# Akar masalahnya: elastisitas

Pool kecil per-pod nggak nyelesain masalah — **pengalinya** (jumlah pod) itu sendiri elastis.

Penskalaan aplikasi  
bebas, elastis

harus dipisah

Jumlah koneksi  
database  
harus stabil & terbatas

Kita butuh sesuatu yang **memisahkan** keduanya.

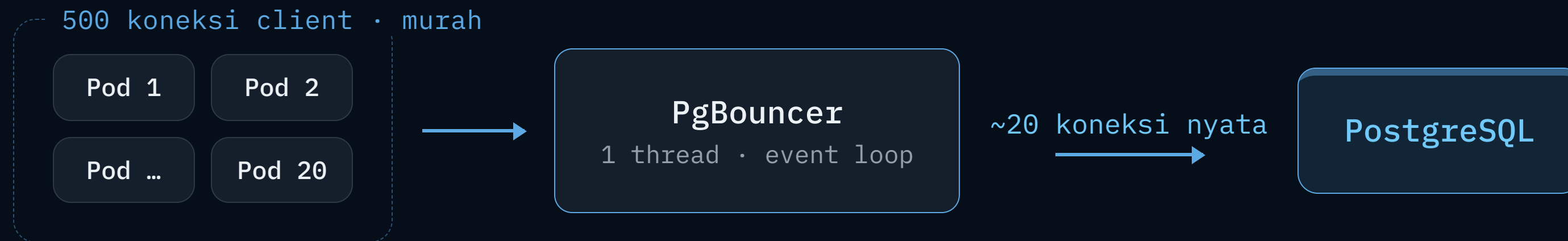
# 4

---

BAGIAN 4

## Solusinya: PgBouncer

# Idenya



Banyak koneksi **murah** di depan → sedikit koneksi **mahal** di belakang.



# ”Kok sanggup pegang 500 koneksi?”

PgBouncer **nggak pakai thread / proses per koneksi.**

- Single-threaded, pakai **event loop** (sama kayak nginx & Redis)
- Koneksi ke PgBouncer = socket + buffer kecil (KB-an)
- Satu thread layani semua koneksi
- Ribuan koneksi idle  $\approx$  **nyaris gratis**

koneksi ke Postgres

**~MB / koneksi**

proses penuh hasil fork

VS

koneksi ke PgBouncer

**~KB / koneksi**

socket + buffer kecil

# Cara dia bagi-bagi koneksi



- Depan: banyak koneksi client (idle)
- Belakang: sedikit koneksi nyata, **dipakai ulang**
- Pool penuh → request **ngantri**, bukan error

# Tiga mode pooling

| Mode        | Koneksi dipinjam selama... | Multiplexing   |
|-------------|----------------------------|----------------|
| Session     | seluruh sesi client        | Rendah         |
| Transaction | satu transaksi             | Tinggi         |
| Statement   | satu statement             | Paling agresif |

Aturan main: **pool size disetel ke concurrency**, bukan ke jumlah koneksi client.



5

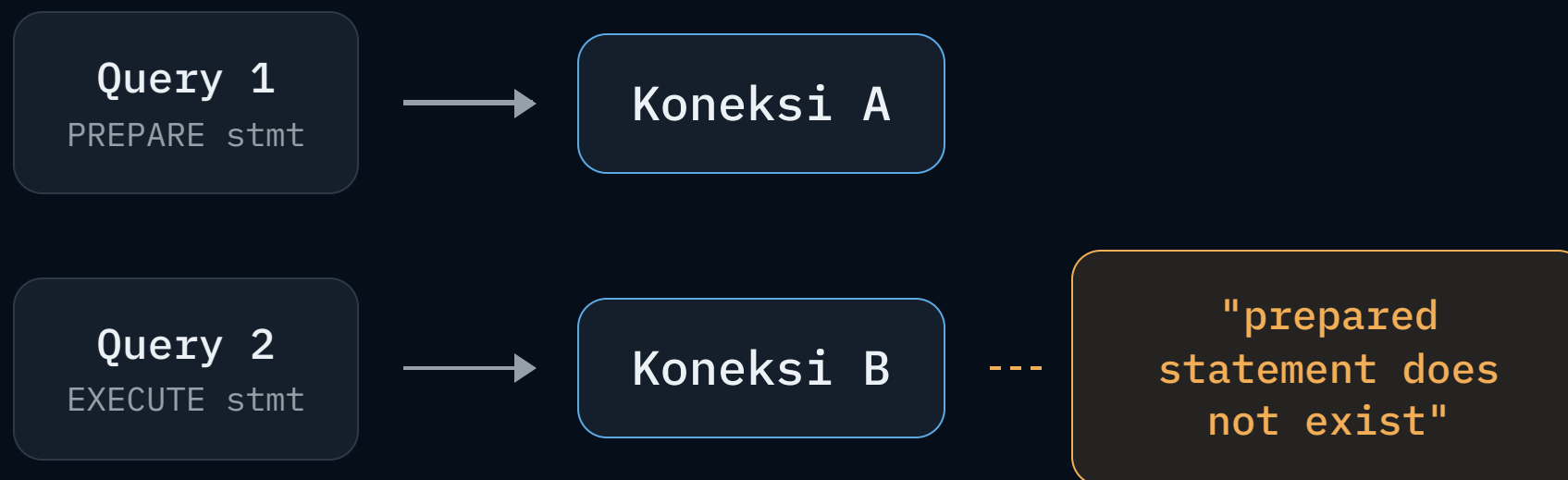
---

BAGIAN 5

# Ada harganya

- Programmer Zaman Now

# Harga dari transaction mode



Koneksi gonta-ganti antar client → **state level-koneksi rusak.**

- Prepared statement, temp table, **SET**, advisory lock
- Semua mengandalkan asumsi "satu sesi = satu koneksi fisik"

MOMEN "KLIK"

# Balik ke bug di awal

- 1 Driver ngaktifin **prepared statement** secara default
- 2 Lewat transaction pooler (6543) yang nggak mendukungnya
- 3 Koneksi gonta-ganti → stmt nggak ketemu → commit gagal **diam-diam**
- 4 RETURNING sempat balik data sebelum commit gagal → keliatan sukses

```
// fix-nya satu baris
```

```
const client = postgres(DATABASE_URL, { prepare: false })
```

Bug-nya **bukan di kode**, tapi di **asumsi soal koneksi**.

# PgBouncer vs Supervisor

|             | PgBouncer                      | Supervisor                       |
|-------------|--------------------------------|----------------------------------|
| Bahasa      | C                              | Elixir (BEAM)                    |
| Concurrency | Single-thread event loop       | Banyak proses ringan + multicore |
| Multicore   | Banyak instance (so_reuseport) | Natural ke banyak core           |
| Cocok buat  | Co-located, low latency        | Multi-tenant, cloud-native       |

Konsep intinya identik: **banyak di depan, sedikit di belakang.**



---

BAGIAN 6

# Ringkasan praktis



# Cara milih jenis koneksi



# Checklist

- Cek dulu apakah pool di app kebesaran (`~core × 2`, sering `~10`)
- Long-running → **direct / session mode**
- Serverless / autoscaling → **transaction pooler** + `prepare: false`
- Pooler harus **terpusat**, bukan sidecar per-pod
- Pooler nyelesain masalah koneksi, bukan throughput
- Throughput mentok → replica, caching, instance lebih gede, sharding

---

PENUTUP

Pooler **memisahkan** penskalaan aplikasi dari jumlah koneksi database.

App boleh scale sebebasnya — koneksi ke Postgres tetap **datar dan terkendali**.

*Dan bug intermittent tadi cuma versi mini dari persoalan yang sama saat scale.*



PROGRAMMER ZAMAN NOW

# Terima kasih

Pertanyaan & diskusi

- PostgreSQL · PgBouncer · Connection pooling